

Ultrasonic Obstacle-avoiding Report



Prepared by: Ahmed Hassan Ibrahim



Table of Contents

- Case Study:**3
- SDLC:**.....3
- Requirements:**.....4
- Space Exploration/ partitioning:**.....4
- System Analysis:**5
 - Use Case Diagram:5
 - Activity Diagram:5
 - Sequence Diagram:6
- System Design:**.....6
 - Trace Simulation:**8
- Code Implementation:**9
 - Output:12



Case Study:

System Description:

A mini car is equipped with an **ultrasonic sensor** and a **DC motor** to control the wheel speed:

- The ultrasonic sensor measures the distance from obstacles in front of the car.
- When the distance drops below a defined threshold, the car stops immediately.
- When the distance is above the threshold, the car accelerates up to a specified maximum speed.
- The system continuously monitors the distance and adjusts the motor speed accordingly.

Assumptions:

- The setup and shutdown of the car are not modeled.
- Car maintenance and hardware calibration are not modeled.
- The ultrasonic sensor never fails.
- The DC motor always functions correctly (no hardware failures).
- The power supply is always available (no power cuts).
- Environmental factors such as uneven surfaces, friction, or sensor noise are not considered.
- The acceleration is ideal (i.e., the motor reaches the target speed instantly without delay).

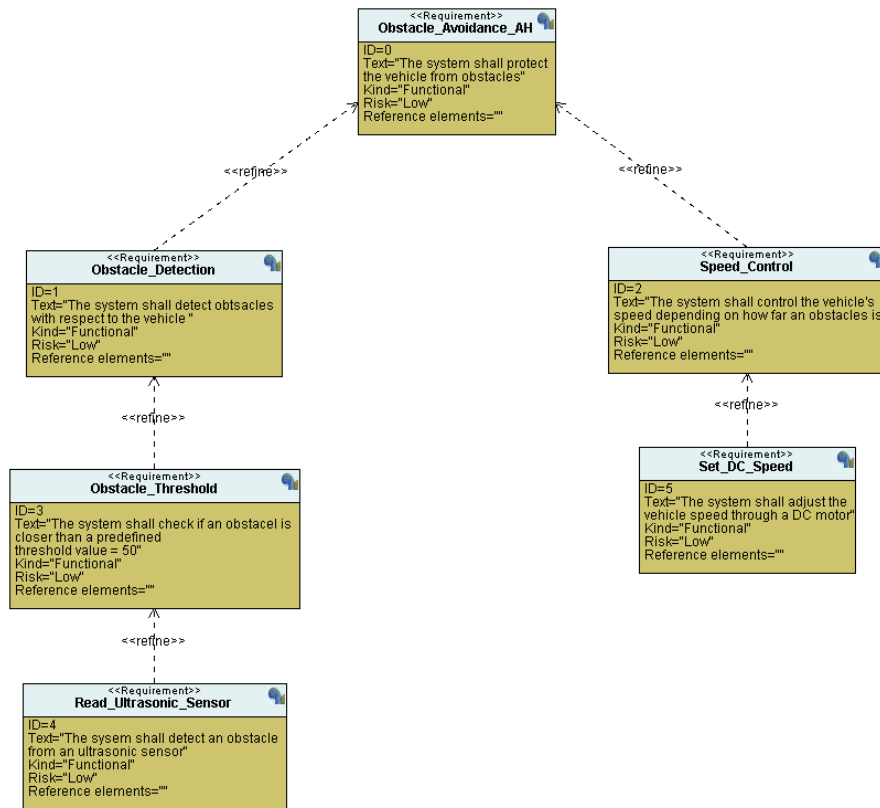
SDLC:

For this project, the **Waterfall Model** will be adopted as the SDLC approach. The development will follow a sequential process, where each phase must be completed before moving to the next:

1. Requirements Analysis
2. Space Exploration/ partitioning
3. System Analysis
4. System Design
5. Implementation (Coding)



Requirements:



Space Exploration/ partitioning:

The project is relatively **simple in scope**, and therefore no significant system partitioning is required. The functionality can be implemented directly on a single microcontroller without the need for distributed control or multiple processing units.

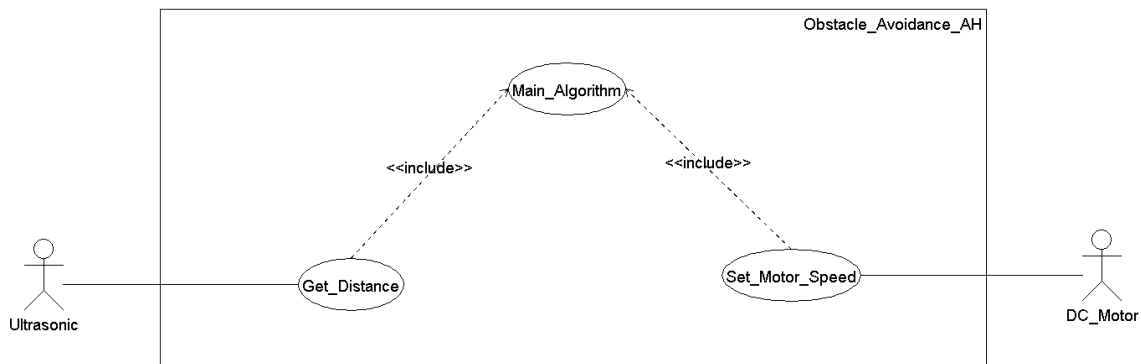
- The **STM32f103c8 microcontroller** will serve as the central controller, handling both sensor input and motor control.
- The **ultrasonic sensor** will provide the distance measurement input.
- The **DC motor** will be controlled directly by the STM32, based on the measured distance relative to the predefined threshold.



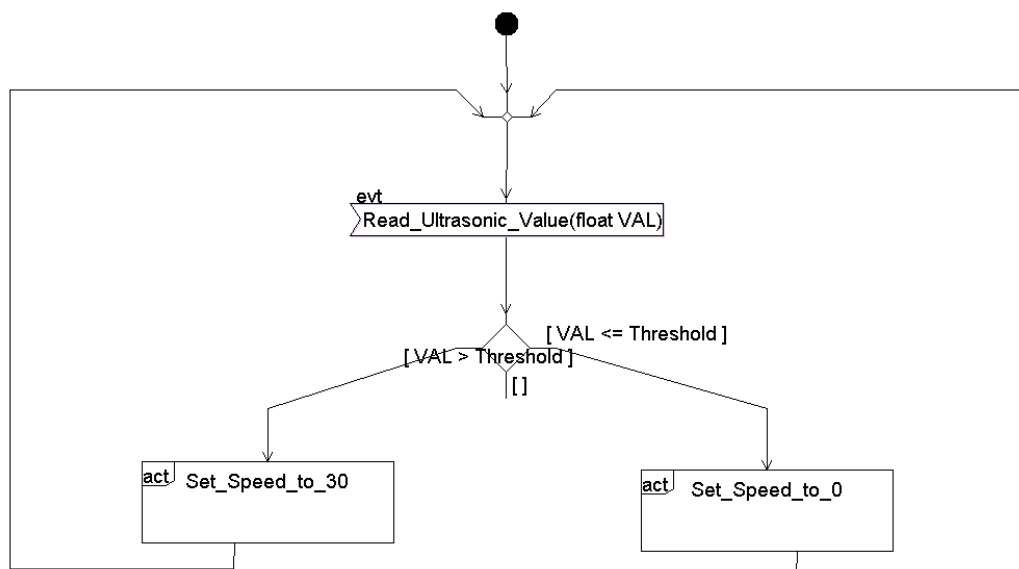


System Analysis:

Use Case Diagram:

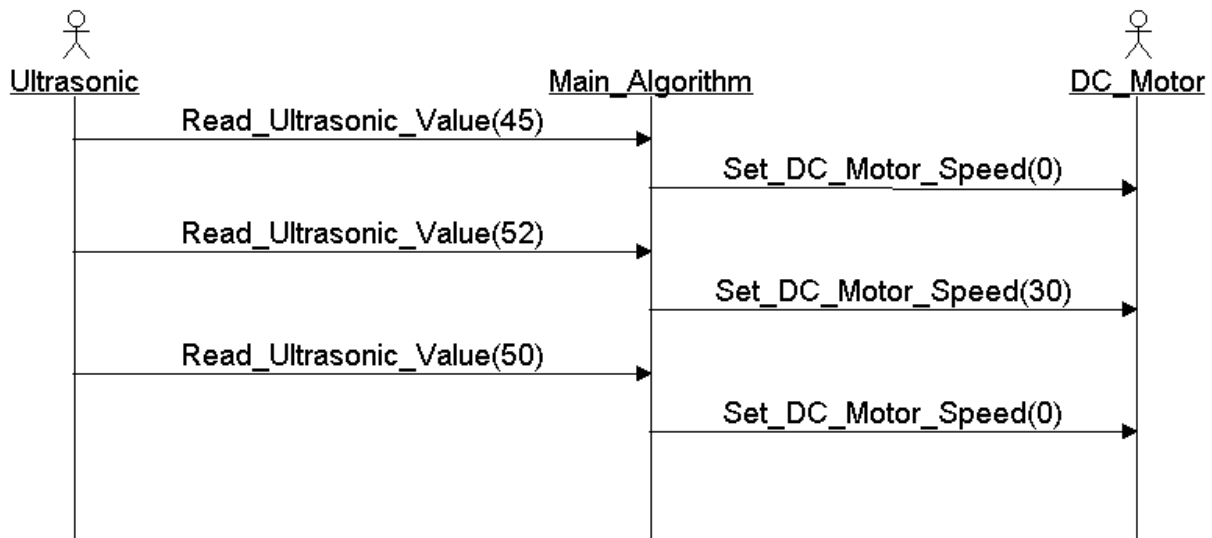


Activity Diagram:

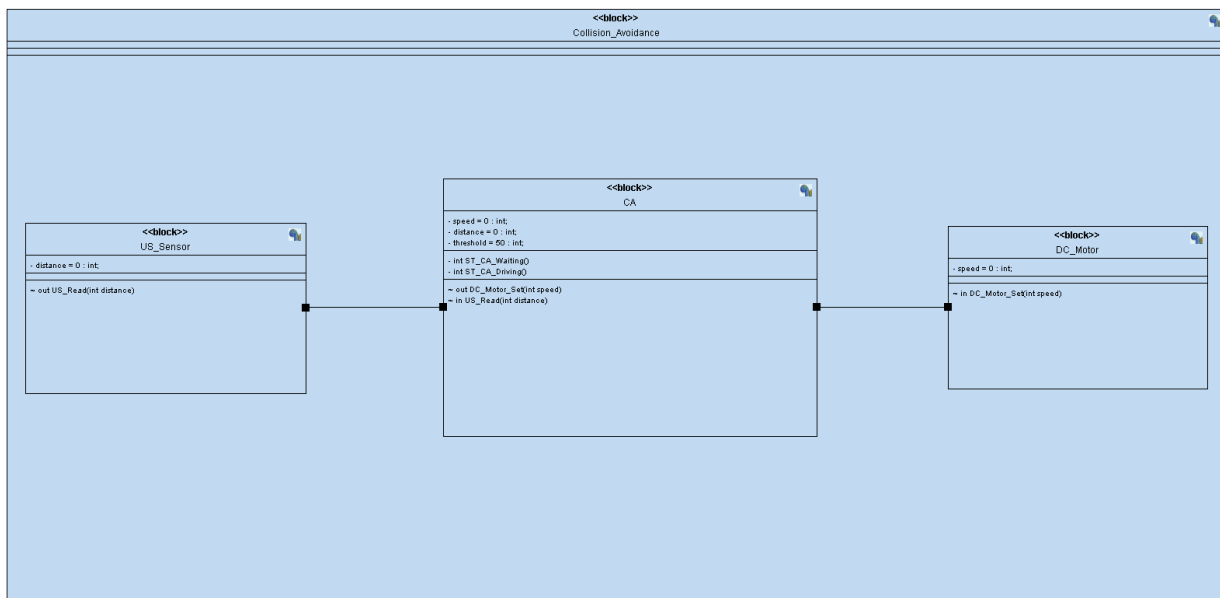




Sequence Diagram:



System Design:



1. Block: US_Sensor

- Purpose:** Measures the distance between the car and an obstacle using an ultrasonic sensor.
- Variables:**
 - Distance – stores the measured distance.
- Signals:**
 - out US_Read(int distance) – sends the measured distance to other modules.

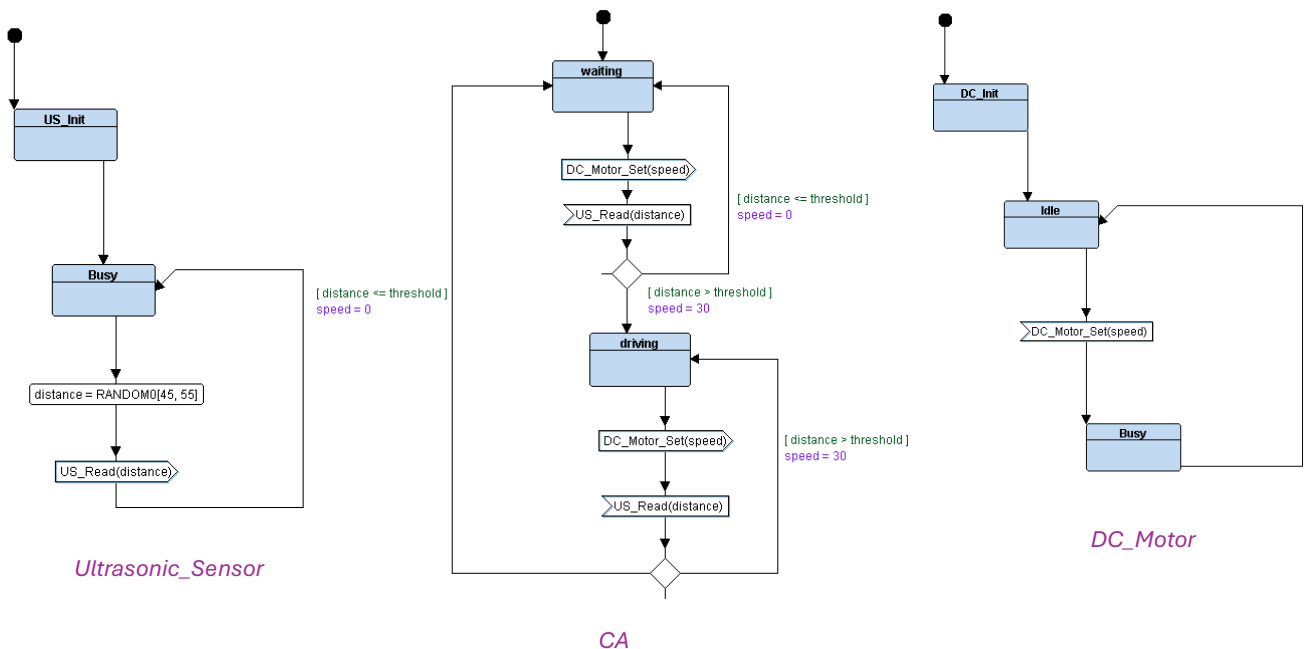


2. Block: CA (Collision Avoidance)

- Purpose:** Implements the decision-making logic to avoid collisions by controlling the car's movement based on distance readings.
- Variables:**
 - Distance – the latest distance value from the ultrasonic sensor.
 - Speed – the car's current speed.
 - Threshold – the minimum safe distance before stopping.
- Signals:**
 - in US_Read(int distance) – receives the measured distance from the ultrasonic sensor.
 - out DC_Motor_Set(int speed) – sends speed commands to the DC motor.

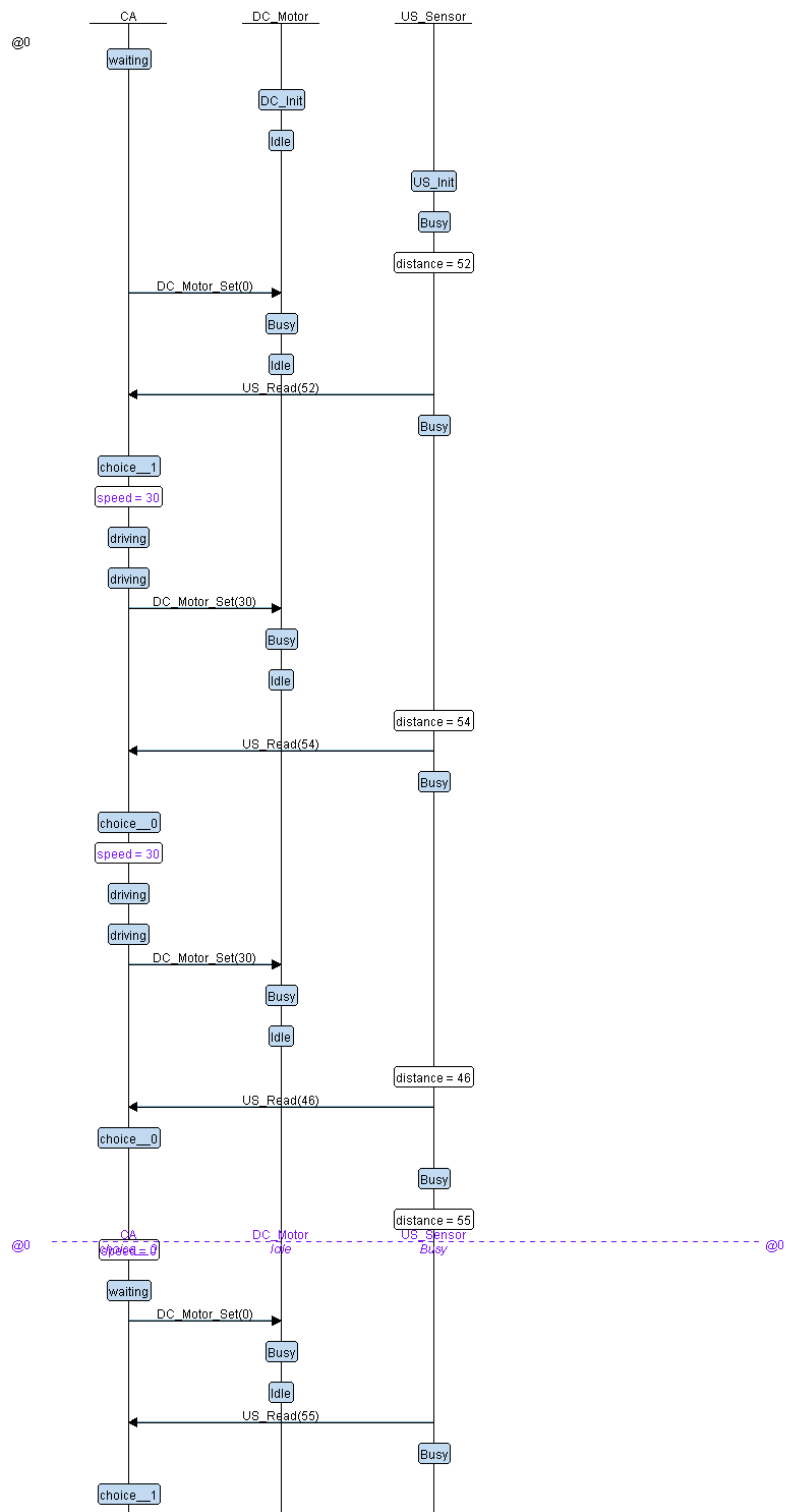
3. Block: DC_Motor

- Purpose:** Controls the speed of the car's wheels.
- Variables:**
 - Speed – current motor speed setting.
- Signals:**
 - in DC_Motor_Set(int speed) – receives speed commands from the CA module.





Trace Simulation:





Code Implementation:

```
C CA.c
h CA.h
C DC.c
h DC.h
C main.c
h state.h
C US.c
h US.h
```

State.h

```
#ifndef STATE_H_
#define STATE_H_

#include "stdio.h"
#include "stdint.h"
#include "stdlib.h"

// Automatic STATE Function Generated
#define STATE_define(_stateFUN_) void ST_##_stateFUN_()
#define STATE(_stateFUN_) ST_##_stateFUN_

// States Connections
void US_Set_Distance(int d);
void DC_Motor_Set(int s);

#endif
```

US.h

```
#ifndef US_H_
#define US_H_

#include "state.h"

// Define states
enum
{
    US_BUSY
};
extern int US_State_ID;
extern void (*US_State)();

// Declare states Functions CA
STATE_define(US_busy);
void US_Init();
int US_Get_Distance_Random(int l, int r, int count);

#endif
```



US.c

```
#include "US.h"

// Variables
static uint32_t US_Distance = 0;
int US_State_ID;
void (*US_State)();

void US_Init()
{
    // Init US Driver
    printf("\nUS_init");
}

STATE_define(US_busy)
{
    // State Name
    US_State_ID = US_BUSY;

    // State Action
    US_Distance = US_Get_Distance_Random(45, 55, 1);

    printf("\nUS_BUSY State: Distance = %d", US_Distance);
    US_Set_Distance(US_Distance);
    US_State = STATE(US_busy);
}

// Function to generate random number in range l and r
int US_Get_Distance_Random(int l, int r, int count)
{
    int i, rand_num;

    for (i = 0; i < count; i++)
        rand_num = (rand() % (r - l + 1)) + l;

    return rand_num;
}
```

DC.h

```
#ifndef DC_H_
#define DC_H_

#include "state.h"

// Define states
enum
{
    DC_IDLE,
    DC_BUSY
};

extern int DC_State_ID;
extern void (*DC_State)();

// Declare states Functions CA
STATE_define(DC_idle);
STATE_define(DC_busy);
void DC_Init();

#endif
```



DC.c

```
● ● ●
#include "DC.h"

// Variables
static uint32_t DC_Speed = 0;

int DC_State_ID;
void (*DC_State)();

void DC_Init()
{
    // Init DC Driver
    printf("\nDC_init");
}

void DC_Motor_Set(int s)
{
    DC_Speed = s;
    DC_State = STATE(DC_busy);
    printf("\nCA----- speed = %d -----> DC", DC_Speed);
}

STATE_define(DC_idle)
{
    // State Name
    DC_State_ID = DC_IDLE;

    // State Action
    // Call PWM to make speed = DC_speed

    printf("\nDC_Idle State: Speed = %d", DC_Speed);
}

STATE_define(DC_busy)
{
    // State Name
    DC_State_ID = DC_BUSY;

    // State Action
    // Call PWM to make speed = DC_speed

    printf("\nDC_Busy State: Speed = %d", DC_Speed);
    DC_State = STATE(DC_idle);
}
```

CA.h

```
● ● ●
#ifndef CA_H_
#define CA_H_

#include "state.h"

// Define states
enum
{
    WAITING,
    DRIVING
};
extern int CA_State_ID;
extern void (*CA_State)();

// Declare states Functions CA
STATE_define(CA_Waiting);
STATE_define(CA_Driving);

#endif
```



CA.c

```
#include "CA.h"

// Variables
static uint32_t CA_Speed = 0;
static uint32_t CA_Distance = 0;
static uint32_t CA_Threshold = 50;
int CA_State_ID;
void (*CA_State)();

void US_Set_Distance(int d)
{
    // Event Check
    CA_Distance = d;
    (CA_Distance <= CA_Threshold) ? (CA_State = STATE(CA_Waiting)) : (CA_State = STATE(CA_Driving));
    printf("\nUS----- distance = %d -----> CA", CA_Distance);
}

STATE_define(CA_Waiting)
{
    // State Name
    CA_State_ID = WAITING;

    // State Action
    CA_Speed = 0;
    DC_Motor_Set(CA_Speed);

    printf("\nCA_Waiting State: Distance = %d    Speed = %d", CA_Distance, CA_Speed);
}

STATE_define(CA_Driving)
{
    // State Name
    CA_State_ID = DRIVING;

    // State Action
    CA_Speed = 30;
    DC_Motor_Set(CA_Speed);

    printf("\nCA_Driving State: Distance = %d    Speed = %d", CA_Distance, CA_Speed);
}
```

main.c

```
#include "CA.h"
#include "DC.h"
#include "US.h"

void setup()
{
    // Init all the drivers
    // Init all IRQ ...
    // Init HAL US_Driver & DC_Driver
    // Init BLOCK
    // Set States Pointers for each Block
    US_Init();
    DC_Init();
    CA_State = STATE(CA_Waiting);
    US_State = STATE(US_busy);
    DC_State = STATE(DC_idle);
}

int main()
{
    setup();

    int i;

    while (1)
    {
        // Call state for each block
        US_State();
        CA_State();
        DC_State();
        for (i = 0; i < 5000; i++);
    }

    return 0;
}
```

Output:

```
US_init
DC_init
US_BUSY State: Distance = 53
US----- distance = 53 -----> CA
CA----- speed = 30 -----> DC
CA_Driving State: Distance = 53    Speed = 30
DC_Busy State: Speed = 30
US_BUSY State: Distance = 54
US----- distance = 54 -----> CA
CA----- speed = 30 -----> DC
CA_Driving State: Distance = 54    Speed = 30
DC_Busy State: Speed = 30
US_BUSY State: Distance = 54
US----- distance = 54 -----> CA
CA----- speed = 30 -----> DC
CA_Driving State: Distance = 54    Speed = 30
DC_Busy State: Speed = 30
US_BUSY State: Distance = 46
US----- distance = 46 -----> CA
CA----- speed = 0 -----> DC
CA_Waiting State: Distance = 46    Speed = 0
DC_Busy State: Speed = 0
US_BUSY State: Distance = 52
US----- distance = 52 -----> CA
CA----- speed = 30 -----> DC
CA_Driving State: Distance = 52    Speed = 30
DC_Busy State: Speed = 30
US_BUSY State: Distance = 50
US----- distance = 50 -----> CA
CA----- speed = 0 -----> DC
```